

Analysis of Activation Functions in Deep Learning: Evaluating Swish Variants on CIFAR-10 Classification

Daniel Adhitthana, Ilyas Rusdi

February 28, 2025

Abstract

This paper evaluates various activation functions in deep learning models trained on the CIFAR-10 dataset. The study focuses on Swish variants, including SiLU, Swish β , E-Swish, P-Swish, and a new proposed Swish variant called L-Swish. We analyze their impact on model performance using ResNet-50, comparing accuracy, precision, recall, and F1-score. Results provide insights into optimal activation choices for image classification.

1 Introduction

Activation functions play a crucial role in deep learning by introducing non-linearity into neural networks, allowing them to learn complex patterns (Ding et al., 2018). Swish-based activations have been proposed as alternatives to traditional functions like ReLU, aiming to enhance model performance by addressing issues such as dying neurons and vanishing gradients (Elfwing et al., 2018; Ramachandran et al., 2017). This study compares Swish variants on the CIFAR-10 dataset, evaluating their impact on classification accuracy, convergence speed, and robustness.

2 Related Works

2.1 Sigmoid-weighted Linear Unit (SiLU)

The Sigmoid-weighted Linear Unit (SiLU) (Elfwing et al., 2018; Ramachandran et al., 2017), also known as the Swish activation function, is a smooth, differentiable, and non-monotonic activation function that has been designed to address some of the limitations of the widely used Rectified Linear Unit (ReLU). Unlike ReLU, which abruptly sets all negative inputs to zero, SiLU smoothly scales the input values using a sigmoid function. This characteristic allows it to maintain small negative values, preventing the loss of valuable gradient information and avoiding the issue of "dying neurons."

SiLU is mathematically defined as:

$$\text{SiLU}(x) = x \cdot \sigma(x) \tag{1}$$

where $\sigma(x)$ represents the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2}$$

This equation means that SiLU applies a gating mechanism where the input x is modulated by the probability-like behavior of the sigmoid function, effectively allowing a more adaptive transformation of the input data.

2.1.1 Properties of SiLU

SiLU possesses several notable properties that distinguish it from other activation functions:

- **Smooth and Differentiable:** Unlike ReLU, which has a discontinuity in its derivative at $x = 0$, SiLU is smooth across the entire real number line due to its unbounded nature. This smoothness ensures better gradient flow and reduces issues related to optimization instability.

- **Non-Monotonicity:** As the input increases, the output does not always increase, allowing for more complex representations and improved feature learning.
- **Self-Gating Mechanism:** The input is adaptively modulated by the sigmoid function, meaning that small positive values are retained while large positive values pass through with minimal change. This behavior makes the function more robust in handling different activation scales.
- **Improved Gradient Flow:** ReLU has zero gradients for negative inputs, which can hinder learning in deep networks. SiLU, in contrast, maintains small gradients even for negative inputs, ensuring that all neurons continue to contribute to learning.

2.1.2 Derivative of SiLU

The derivative of an activation function plays a crucial role in backpropagation, as it determines how gradients are propagated during training. The derivative of SiLU can be computed as follows:

$$\frac{d}{dx}\text{SiLU}(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x)) \quad (3)$$

This derivative consists of two terms:

- The first term, $\sigma(x)$, represents the sigmoid gating function.
- The second term, $x\sigma(x)(1 - \sigma(x))$, introduces a modulated gradient component.

Unlike ReLU, which has a zero derivative for all negative inputs, SiLU maintains nonzero gradients throughout, ensuring stable and continuous gradient updates.

Training Results:

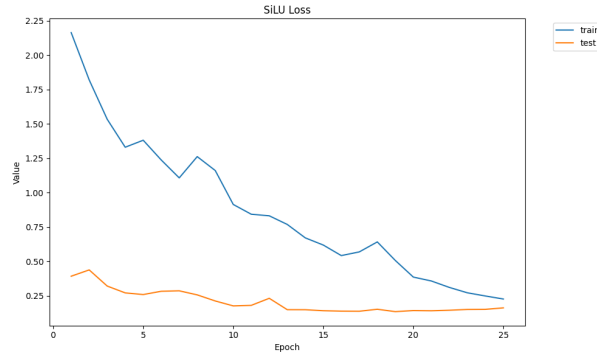


Figure 1: Training loss and testing loss for SiLU activation function.

2.2 Swish β

2.2.1 Properties of Swish β

Swish β has an additive property from Swish:

- **Improved Gradient Flow:** The introduction of β allows fine-tuning of the activation, improving the gradient flow and preventing the network from getting stuck in regions where gradients are too small to drive meaningful updates.

2.2.2 Derivative of Swish β

The derivative of an activation function plays an essential role in determining how gradients are propagated during backpropagation. For Swish β , the derivative can be computed as follows:

$$\frac{d}{dx}\text{Swish}_\beta(x) = \sigma(\beta x) + x \cdot \sigma'(\beta x) \cdot \beta \quad (4)$$

This derivative can be broken down into two parts:

- The first term, $\sigma(\beta x)$, represents the sigmoid function that governs the gating mechanism.
- The second term, $x \cdot \sigma'(\beta x) \cdot \beta$, adjusts the gradient based on the parameter β and the input value x . This term modulates the sensitivity of the gradient, allowing Swish β to adapt its learning behavior dynamically.

This derivative ensures that Swish β retains nonzero gradients even for negative inputs, unlike ReLU. The presence of the learnable parameter β further enhances the gradient flow, improving the model's ability to learn from the data.

Training Results:

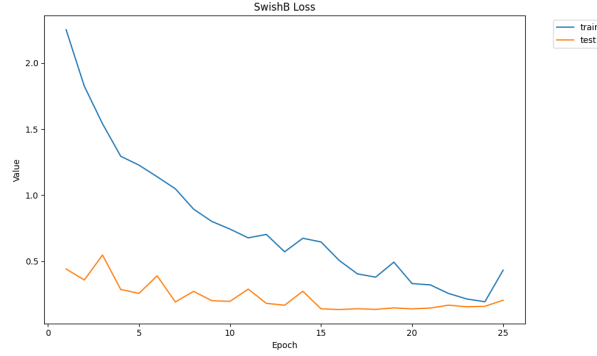


Figure 2: Training loss and testing loss for Swish β activation function.

2.3 E-Swish

2.3.1 Properties of E-Swish

E-Swish (Alcaide, 2018) modifies the traditional Swish activation function by introducing a scaling factor β , which allows faster learning. This scaling factor plays a crucial role in controlling the learning dynamics of the model. By adjusting β , we can influence the speed at which the model learns, particularly in the early stages of training. Specifically, increasing β speeds up the learning process by making the function more sensitive to changes in the input, but it can also cause the gradients to become too large, leading to instability. Therefore, the parameter β needs to be chosen carefully to balance rapid learning with gradient stability.

- **Increased Sensitivity with Scaling:** The scaling factor β effectively adjusts the steepness of the Swish function. A larger β results in a steeper curve, allowing the model to learn faster by emphasizing the differences between positive and negative inputs. However, when β becomes too large, there is a risk of gradients exploding, which can destabilize training.
- **Control Over Learning Dynamics:** The introduction of the learnable parameter β in E-Swish allows more control over the activation function's behavior. By tuning β , the network can be adapted to different learning rates and data distributions. This makes E-Swish highly flexible and capable of adapting to diverse datasets, enhancing feature learning and training efficiency.
- **Prevents Exploding Gradients:** Despite the scaling factor β increasing sensitivity, the range $1 \leq \beta \leq 2$ is chosen to prevent the gradients from exploding. This range ensures that the function remains stable while still providing the benefits of faster learning, making it a practical choice for deep neural networks.

2.3.2 Derivative of E-Swish

To compute the derivative of E-Swish, we take the derivative of the function with respect to x :

$$\frac{d}{dx} \text{ESwish}(x) = \beta \cdot \sigma(x) + \beta x \cdot \sigma'(x) \quad (5)$$

This derivative has two components:

- The first term, $\beta \cdot \sigma(x)$, represents the effect of the scaling factor on the sigmoid function.

- The second term, $\beta x \cdot \sigma'(x)$, incorporates the gradient of the sigmoid function, modulated by the scaling factor β , adjusting the steepness of the activation.

The derivative ensures that E-Swish maintains smooth and continuous gradients across all input values. By adjusting β , the model can control the magnitude of the gradient, allowing for efficient backpropagation and stable training. The scaling factor β plays a critical role in balancing the trade-off between learning speed and gradient stability, ensuring that the network can converge effectively.

Training Results:

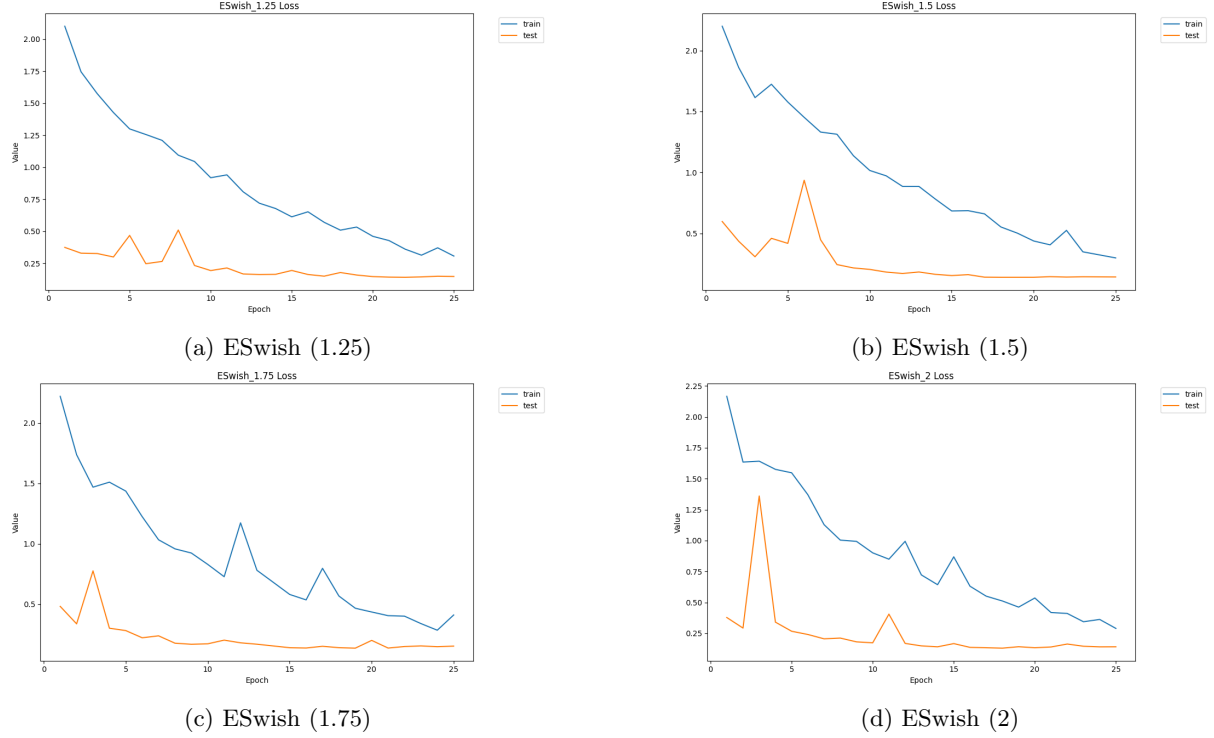


Figure 3: Training loss for different variations of E-Swish with varying values of β .

2.4 P-Swish

2.4.1 Properties of P-Swish

P-Swish (Mercioni & Holban, 2020) is a modification of the Swish activation function that uses piecewise functions and learnable parameters α , β , and δ . Its parametric nature allows P-Swish to have greater flexibility in handling different data distributions.

P-Swish is defined as the following:

$$\text{PSwish}(x) = \begin{cases} a \cdot \sigma(\delta x) & \text{if } x \leq \beta \\ x & \text{if } x > \beta \end{cases} \quad (6)$$

For our experiments, we follow the same P-Swish variants tested on the original paper (Mercioni & Holban, 2020):

- **PSwish (None):** In the default configuration, all parameters (α , β , δ) are initialized to 1, 1, and 0 in order. This means that P-Swish behaves like the standard Swish, with output $x \cdot \sigma(x)$ for $x \leq 0$ and identity for $x > 0$.
- **PSwish (ab):** Both parameters α and β are learnable during training. The parameter α is responsible for scaling the output for $x \leq \beta$, while β serves as the threshold that separates the sigmoid part from the identity part.
- **PSwish (a):** Only the parameter α is learnable, while β and δ are constants. This configuration is useful when we want to modify the scaling of the output for the sigmoid part (when $x \leq \beta$), but

we still want to maintain the default threshold β and steepness δ . This allows the model to focus on adjusting the scale without needing to change the threshold or steepness behavior.

- **PSwish (ad):** The parameters α and δ are learnable during training, while β is constant. Allowing α and δ to be learned gives more control over the scaling of the output for the sigmoid part and the steepness of the sigmoid function. This allows the model to adjust how the sigmoid modifies the input, without changing the threshold β .

2.4.2 Derivative of P-Swish

To compute the gradients during backpropagation, we need to consider the derivative of P-Swish, which is computed piecewise depending on the value of x . Its derivative is as follows:

$$\frac{d}{dx}\text{PSwish}(x) = \begin{cases} \alpha \cdot \sigma(\delta x) + \alpha x \cdot \delta \sigma'(\delta x), & \text{if } x \leq \beta \\ 1, & \text{if } x > \beta \end{cases} \quad (7)$$

This derivative consists of two main components:

- For $x \leq \beta$, the derivative has two components:
 - The first component, $\alpha \cdot \sigma(\delta x)$, represents the gradient of the sigmoid component that has been scaled by α .
 - The second component, $\alpha x \cdot \delta \sigma'(\delta x)$, represents the contribution of the slope of the sigmoid, adjusted by parameter δ .
- For $x > \beta$, the derivative is 1 since the function is identity.

Training Results:

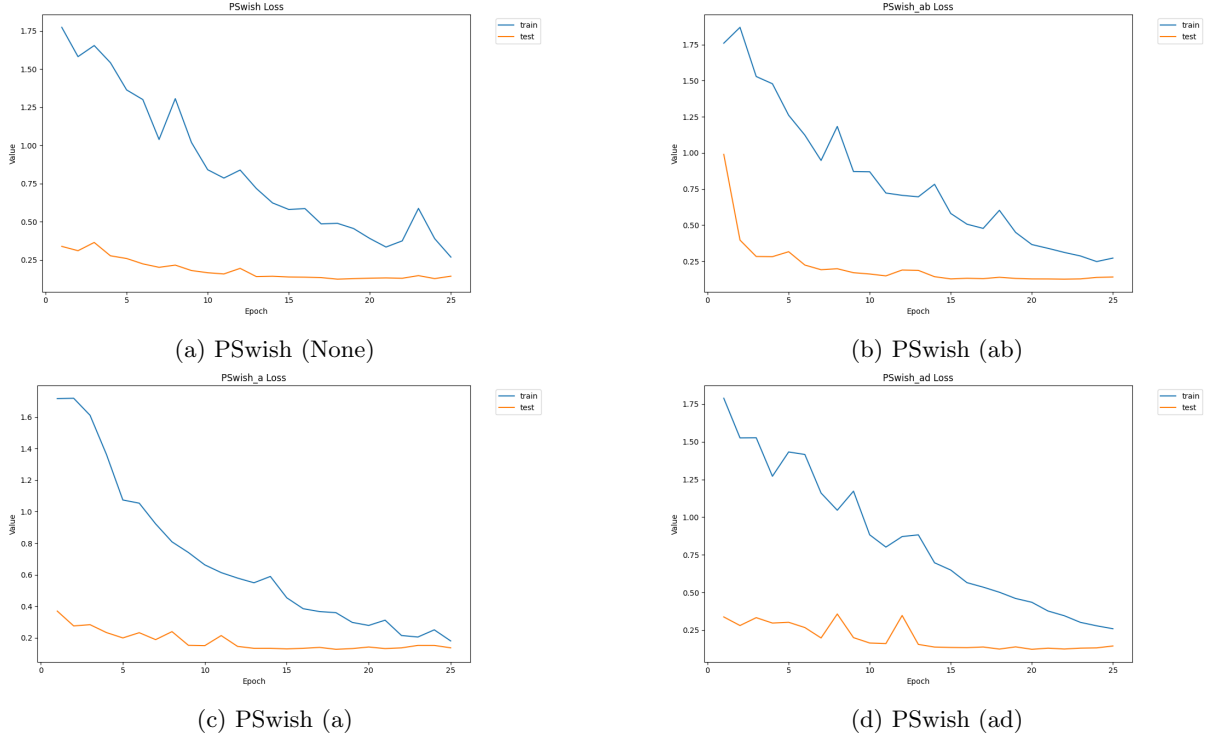


Figure 4: Training loss for different variations of P-Swish with varying combinations of learnable parameters.

2.5 L-Swish

We propose a parametric swish variant that utilizes the nature of parametric functions (Pusztaházi et al., 2024) and bias, called L-Swish. We hypothesize that by adding bias in the sigmoid term, the model can adjust towards the minima faster which can lead to faster generalization.

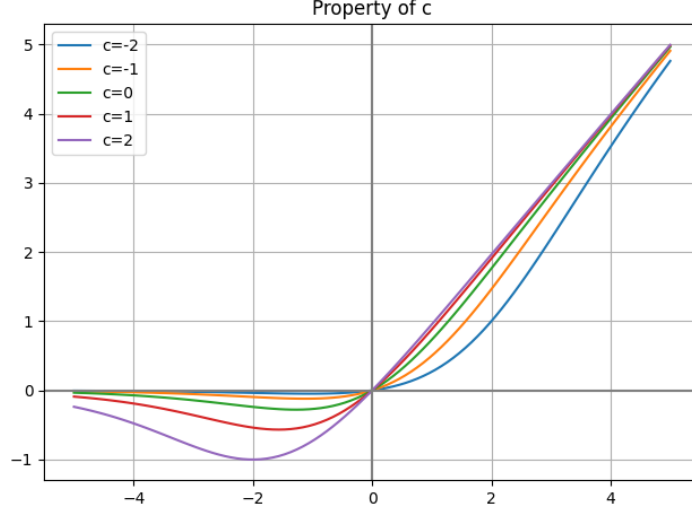


Figure 5: Property of bias in sigmoid term

The function looks like the following:

$$\text{LSwish}(x) = ax \cdot \sigma(bx + c) \quad (8)$$

where a, b, c are learnable parameters. Each parameter is initialized as 1, 1, and 0 in order.

Training Results:

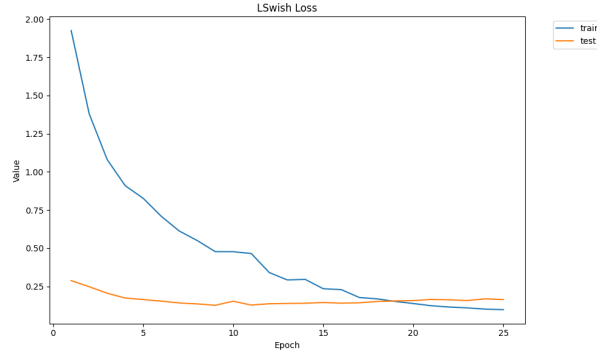


Figure 6

3 Experiments

3.1 CIFAR-10 Dataset

The CIFAR-10 dataset (Krizhevsky, Hinton, et al., 2009) consists of 60,000 32x32 color images across 10 classes. The images are then split into 50,000 for training and 10,000 for testing. We also apply data preprocessing, including normalization and data augmentation techniques such as horizontal flips, to enhance generalization.

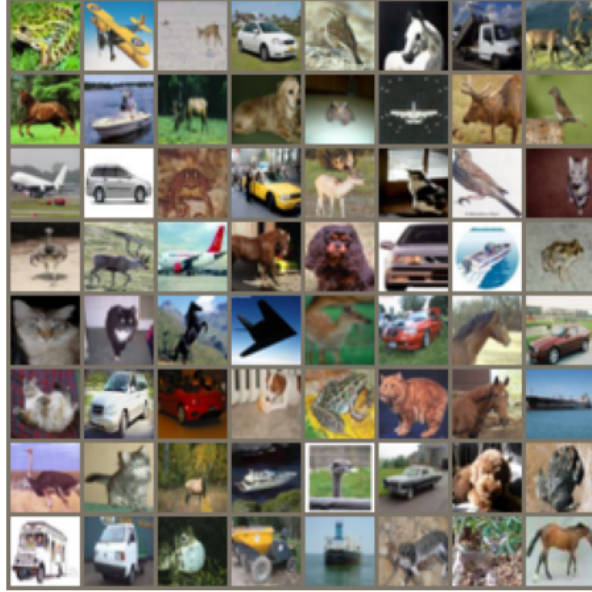


Figure 7: A random sample of images from the CIFAR-10 dataset

3.2 Model Architecture

We employ ResNet-50 (He et al., 2016) as the baseline architecture, modifying the ReLU activation functions into the swish variants to assess their impact. The network consists of convolutional layers, batch normalization, residual connections, and fully connected layers.

3.3 Training and Evaluation

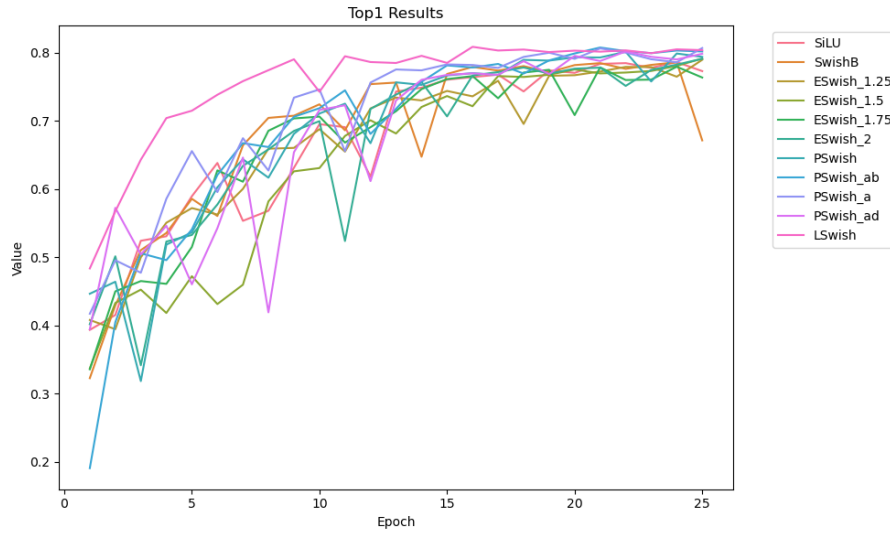
Each model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.001 and cross-entropy loss. Evaluation metrics include Top-1 and Top-5 accuracy, precision, recall, and F1-score. Experiments are conducted across multiple runs to ensure statistical significance.

3.4 Results

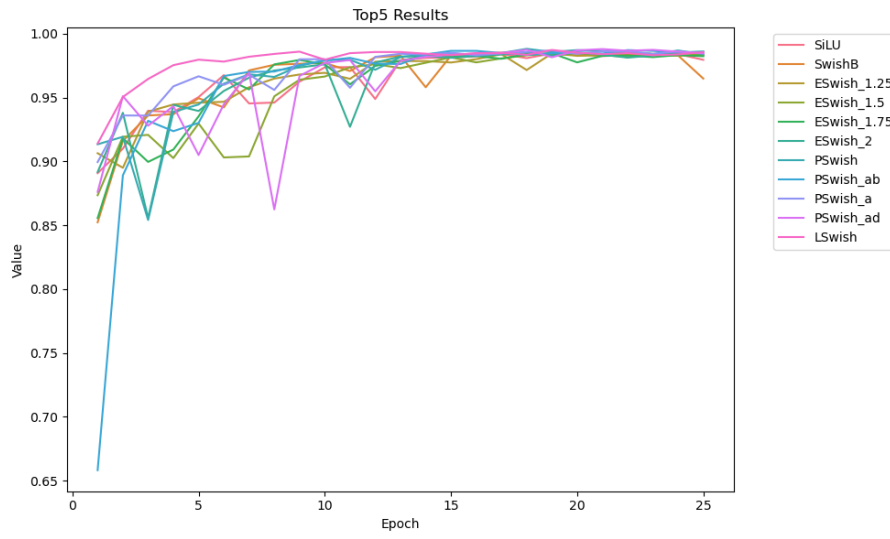
Our experiments show that P-Swish (a) achieved the best performance on almost all metrics, while L-Swish performs second best on all metrics

Activation	Top-1	Top-5	Precision	Recall	F1
Swish	77.28	97.96	77.85	77.28	77.41
Swish β	67.12	96.49	67.29	67.12	67.00
E-Swish ($\beta = 1.25$)	78.94	98.34	79.41	78.94	79.07
E-Swish ($\beta = 1.5$)	79.12	98.32	78.99	79.12	79.01
E-Swish ($\beta = 1.75$)	76.34	98.24	76.39	76.34	76.25
E-Swish ($\beta = 2$)	79.09	98.60	79.20	79.09	79.10
P-Swish (None)	79.37	98.40	79.12	79.37	79.11
P-Swish (ab)	80.16	98.59	80.29	80.16	80.17
P-Swish (a)	80.66	98.46	80.89	80.66	80.71
P-Swish (ad)	79.82	98.47	80.36	79.82	79.95
L-Swish	80.35	98.56	80.54	80.35	80.34

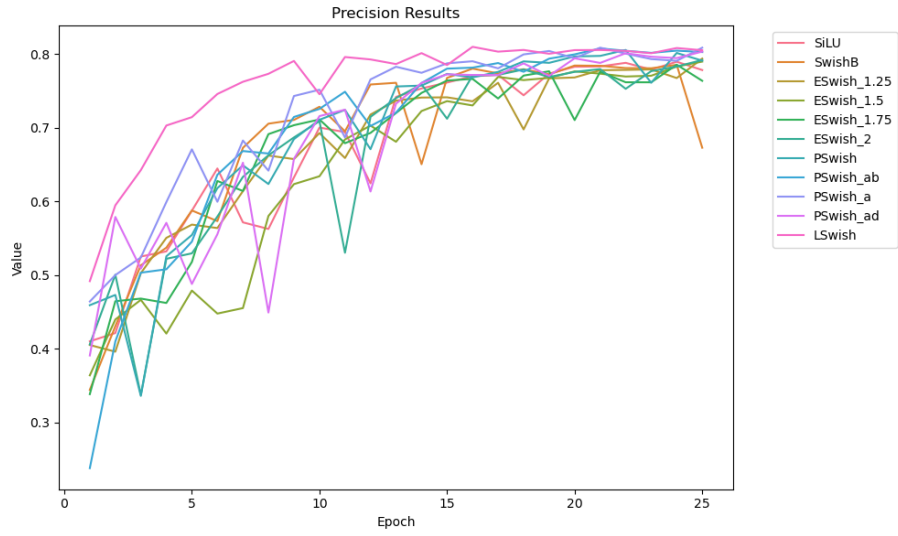
Figure 8: Experiment results



(a) Top-1 Accuracy



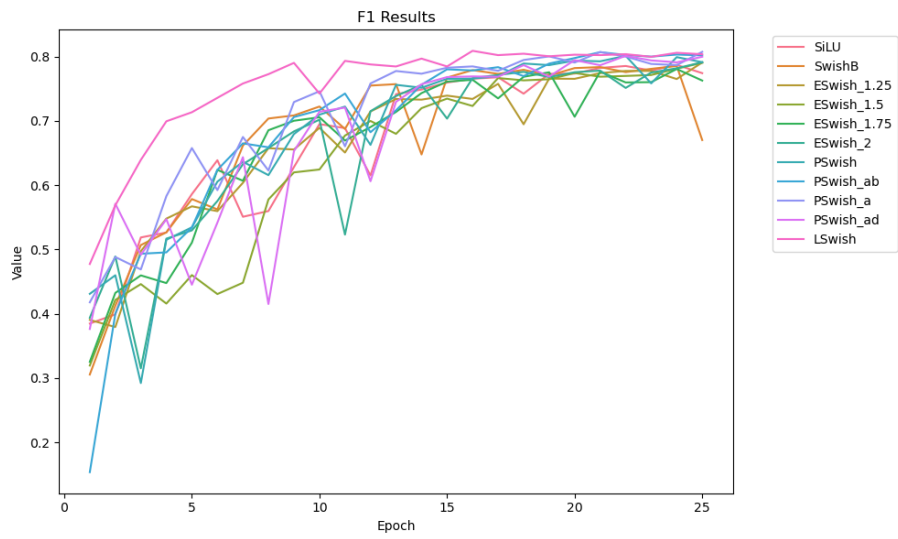
(b) Top-5 Accuracy



(a) Precision



(b) Recall



(c) F1-Score

4 Conclusion

This study highlights the effectiveness of Swish-based activation functions on image classification. Our experimental results demonstrate that L-Swish can outperform several swish variants in terms of faster convergence due to the bias at the sigmoid term. However, the final result shows L-Swish being outperformed by P-Swish in almost every metric. This might be caused by overfitting as seen in the loss plot. Future research could further optimize L-Swish and other swish variants for several applications.

References

- Alcaide, E. (2018). E-swish: Adjusting activations to different network depths. *arXiv preprint arXiv:1801.07145*.
- Ding, B., Qian, H., & Zhou, J. (2018). Activation functions and their characteristics in deep neural networks. *2018 Chinese control and decision conference (CCDC)*, 1836–1841.
- Elfwing, S., Uchibe, E., & Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107, 3–11.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.
- Krizhevsky, A., Hinton, G., et al. (2009). Learning multiple layers of features from tiny images.
- Mercioni, M. A., & Holban, S. (2020). P-swish: Activation function with learnable parameters based on swish activation function in deep learning. *2020 International Symposium on Electronics and Telecommunications (ISETC)*, 1–4.
- Pusztaházi, L. S., Eigner, G., & Csiszár, O. (2024). Parametric activation functions for neural networks: A tutorial survey. *IEEE Access*.
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Swish: A self-gated activation function. *arXiv preprint arXiv:1710.05941*, 7(1), 5.